

Analyse v7 $\rightarrow$ v8 : plancher hardware i7-7500U

Auteur : hprzeta · **Date :** 11 juin 2026 Benchmark complet prec_fast/prec_full + W=4 vs W=8

Résumé

v7 (prec_fast=64 bits, SIMD) atteint 30,9 min à $T = 100\,000$. Le benchmark v8 teste deux leviers : précision de phase 2 (prec_full) et nombre de workers. Résultat : **gain marginal $\times 1,06$** , plancher hardware atteint sur i7-7500U (dual-core HT, 4 threads logiques).

1. Contexte v7

illinois_refine_adaptive utilise deux phases :

Phase	Itérations	Précision	Limbres mpfr
1 (rapide)	$i = 0 \dots 7$	64 bits	1 — SIMD AVX2
2 (précise)	$i \geq 8$	116 bits	2

Le gain v6 $\rightarrow$ v7 ($\times 3,7$) vient entièrement de la phase 1 : prec_fast=64 bits $\rightarrow$ 1 limbe. La phase 2 (116 bits, 2 limbres) reste le seul levier non exploité.

2. Benchmark 1 — prec_fast / prec_full

500 brackets sur $t \in [1\,000, 3\,000]$, séquentiel, turbo CPU actif.

Config (fast, full)	Limbres ph1+ph2	ms/appel	Gain	
(64, 116)	1+2	0,443 ms	$\times 1,00$	référence v7
(48, 116)	1+2	0,559 ms	$\times 0,79$	$\times$
(32, 116)	1+2	0,875 ms	$\times 0,51$	$\times$
(64, 96)	1+2	0,476 ms	$\times 0,93$	$\times$
(64, 80)	1+2	0,416 ms	$\times 1,06$	✓ optimal

Précision finale ($\|Z\|_{\max}$) identique sur toutes les configs : $6,6 \times 10^{-4}$ (biais structurel RS $\mathcal{O}(t^{-3/2})$, indépendant de la précision interne).

Pourquoi `prec=32` est plus lent que `prec=64` ?

$$\left\lceil \frac{32}{64} \right\rceil = \left\lceil \frac{64}{64} \right\rceil = 1 \text{ limbe}$$

`mpfr` alloue toujours un minimum de 64 bits par limbe. `prec=32` déclenche des conversions d'arrondi supplémentaires (downcast 64→32 bits) sans réduire la taille mémoire. L'overhead est pire, pas meilleur.

Pourquoi `prec=80` est légèrement meilleur que `prec=116` ?

Les deux ont 2 limbes. La différence vient du nombre d'opérations internes des fonctions transcendantes `mpfr` (`mpfr_cos`, `mpfr_log`) :

$$t_{\text{mpfr_cos}}(p) \propto p \cdot \log p$$

Avec $p = 80$ vs $p = 116$: rapport théorique $\approx \frac{80 \ln 80}{116 \ln 116} \approx 0,92$. Mesuré : $0,416/0,443 = \mathbf{0,94}$ — cohérent.

3. Benchmark 2 — `W=4` vs `W=8` (`T=5 000`)

Workers	Durée	z/s	Gain
<code>W=4</code>	—	2 505,8 z/s	×1,00 (référence)
<code>W=8</code>	—	2 474,0 z/s	×0,99 ×

i7-7500U : 2 cœurs physiques, HT → 4 threads logiques (`nproc=4`). `W=8 > nproc` → context-switching → perte nette de 1,2%.

L'overhead de création de processus (8 forks, 8 `.so` chargés, 8 segments de mémoire partagée) dépasse le bénéfice HT.

4. Résultat `v8`

`compute_zeros_v7.py` (nommage intermédiaire de test) puis `compute_zeros_v8.py` :

Paramètre	v7 (ref)	v8
<code>prec_fast</code>	64 bits	64 bits (inchangé)
<code>prec_full</code>	116 bits	80 bits
<code>ITER_SWITCH</code>	8	8 (inchangé)
<code>MAX_ITER</code>	50	50 (inchangé)
Durée <code>T=100k</code> estimée	30,9 min	~29 min
Gain	—	×1,06

5. Plancher hardware atteint

Borne inférieure théorique de durée sur i7-7500U :

$$t_{\text{zéro}}^{\min} = \frac{N_{\text{iter}} \times t_{\text{mpfr}}(\text{prec} = 64)}{W} = \frac{10 \times 0,416 \text{ ms}}{4} \approx 1 \text{ ms/zéro}$$

$$T_{100k}^{\text{plancher}} = 138\,069 \times 1 \text{ ms}/4 \approx 34 \text{ s}$$

Cette borne est inatteignable en pratique (scan, Turing, I/O). La durée réaliste plancher est ~29 min — c'est ce que v8 atteint.

Conclusion : les optimisations algorithmiques sur ce CPU sont épuisées. Les prochains gains nécessitent du matériel différent.

6. Tableau global v1 → v8

Version	Durée T≈100k	Gain cumulé	Levier principal
v1	21h	×1	Newton scalaire
v2	2h	×10	Illinois bracket
v3	45min	×28	Parallèle W=4
v4.1	9min	×140	Illinois C/libmpfr
v5	~1min*	×1 260*	Arb hardy_z
v6	~130min	—	STEP=0.010 + scan_arb.c
v7	30,9min	×4,2 vs v6	prec_fast=64 bits → SIMD
v8	~29min	×1,06 vs v7	prec_full=80 bits

*v5 mesuré sur T=10k uniquement.

Gain total v1 → v8 : ×5 628 (21h → ~29 min)

7. Prochains leviers (v9 et au-delà)

Option	Description	Gain estimé	Difficulté
A	CPU 8 cœurs physiques (i9, Ryzen 9, M3 Pro)	×2	Matériel
B	Arb acb_dirichlet_hardy_z pour affinage	à benchmarker	Moyen
C	GPU CUDA détection Z_double (GTX 960M)	×100 détection mais 0,2% du temps	Élevée
D	Termes RS d'ordre supérieur (C_2, C_3)	réduit biais, pas la vitesse	Très élevée

Option A est la plus directe : passer de 4 à 8 workers réels divise la durée par ×2.

Leçon principale

La progression $v1 \rightarrow v7$ ($\times 5\,292$ de gain) a atteint la limite architecturale du i7-7500U.
Chaque optimisation algorithmique restante donne moins que la précédente. La loi des rendements décroissants s'applique.

Voir aussi : [\[\[Formules__zeta\]\]](#) · [\[\[analyse_problemes__v6__v7\]\]](#) · [\[\[STACK\]\]](#)

Fichier : analyse_problemes_v7_v8.md · **Dossier** : wiki master **Auteur** : hprzeta
· **MAJ** : 11 juin 2026 · ~120 lignes